Stochastic Gradient Descent

- Error functions based on maximum likelihood for a set of independent observations comprise a sum of terms, one for each data point

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}).$$

- The most widely used training algorithms for large data sets are based on a sequential version of gradient descent known as **stochastic gradient descent** or **online gradient descent**.
- which updates the weight vector based on one data point at a time

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n\left(\mathbf{w}^{(\tau)}\right).$$

- A complete pass through the whole training set is a training **epoch**.

Stochastic Gradient Descent

- SGD handles redundancy in the data much more efficiently.
- Take a data set and double its size by duplicating every data point.
- This simply multiplies the error function by a factor of 2 and so is equivalent to using the original error function.
- Batch methods require double computating effort to evaluate the gradient, whereas SGD will be unaffected.
- SGD is possibile to escape from local minima, since a stationary point w.r.t. the error function for the whole data set will generally NOT be a stationary point for each data point individually.

Mini-batches

- Gradient computed from *a single data point* provides a very *noisy* estimate of the gradient computed on the *full data set*.
- An intermediate approach (Batch vs Stochastic), a *small subset* of data points, called a **mini-batch**, is used to evaluate the gradient at each iteration.

$$\mathbf{w} = \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$$

Mini-batches

The optimum size for the mini-batch

- error in computing the mean from N samples is given by $\sigma/\sqrt{N}$: diminishing returns in estimating the true gradient from increasing the batch size.
- make efficient use of the hardware arch. on which the code is running

Sequential Correlations

- the constituent data points should be chosen randomly from the data set

Parameter initialization

- The specific initialization can have a significant effect on
  - how long it takes to reach a solution
  - and on the generalization performance of the resulting trained network

symmetry breaking

The distribution used to initialize the weights is typically

- either a uniform distribution in the range $[-\varepsilon, \varepsilon]$
- or a zero-mean Gaussian of the form $\mathcal{N}(0, \varepsilon^2)$.

Error Backpropagation

Error Backpropagation

- Our goal in this section is to find an efficient technique for evaluating the gradient of an error function $E(\mathbf{w})$ for a feed-forward neural network.
- this can be achieved using **a local message passing scheme** in which information is sent alternately forwards and backwards through the network
- this is known as **error backpropagation**, or sometimes simply as **backprop**.

Error Backpropagation: the Nature of the Training Process

- Training involves sequence of steps to **adjust weights**, and there are two stages in each step

1. To evaluate the derivatives of the error function w.r.t. weights.
    - the backpropagation is important in providing a computationally efficient method for evaluating such derivatives.
    - errors are propagated backwards through the network.
2. Derivatives are then used to compute the adjustments to be made.
    - The simplest such technique involves gradient descent.

Error Backpropagation: the Nature of the Training Process

- The two stages are distinct.
- The first stage, namely **the propagation of errors backwards through the network** in order to evaluate derivatives, can be applied to many other kinds of network and not just the multilayer perceptron.
- It can also be applied to error functions other that just the simple sum-of-squares, and to the evaluation of other derivatives such as the Jacobian and Hessian matrices
- The second stage of **weight adjustment** using the calculated derivatives can be tackled using a variety of optimization schemes, many of which are substantially more powerful than simple gradient descent.

Evaluation of error-function derivatives

- Error functions defined by maximum likelihood for a set of i.i.d. data, comprise a sum of terms, one for each data point in the training set

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}).$$

- A linear model where outputs y_k are linear combinations of the input variables x_i

$$y_k = \sum_i w_{ki} x_i$$

- together with an error function that, for a particular input pattern n,

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2$$

Evaluation of error-function derivatives

- Consider the error function of one data point, its gradient w.r.t. a weight w_{ji} is given by

$$\frac{\partial E_n}{\partial w_{ji}} = (y_{nj} - t_{nj})x_{ni}$$

- which can be interpreted as a "local" computation involving the product of
  - an "error signal" $y_{nj} - t_{nj}$ associated with the output end of the link w_{ji}
  - the variable x_{ni} associated with the input end of the link.

Evaluation of error-function derivatives

- In a general feed-forward network, each unit computes a weighted sum of its inputs of the form

$$a_j = \sum_i w_{ji} z_i \qquad \text{(weighted sum)}$$

- where z_i is the activation of a unit, or input, that sends a connection to unit j, and w_{ji} is the weight associated with that connection.
- The sum is transformed by a nonlinear activation function $h(\cdot)$ to give the activation z_j of unit j in the form

$$z_j = h(a_j) \qquad \text{(activation)}$$

Evaluation of error-function derivatives

- For each pattern in the training set, we suppose that
  - we have supplied the corresponding input vector to the network
  - and calculated the activations of all of the hidden and output units in the network by successive application of the two above equations.
- This process is often called **forward propagation**
  - because it can be regarded as a forward flow of information through the network.

Evaluation of error-function derivatives

- Let us consider the evaluation of the derivative of E_n w.r.t. a weight w_{ji}.
- The outputs of the various units will depend on the input pattern n.
- However, in order to keep the notation uncluttered, we omit the subscript n from the network variables.
- First we note that E_n depends on the weight w_{ji} only via the summed input a_j to unit j.

Evaluation of error-function derivatives

- We can therefore apply the chain rule for partial derivatives to give

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}} \qquad \text{(chain)}$$

- We introduce a useful notation for **errors**

$$\delta_j \triangleq \frac{\partial E_n}{\partial a_j}$$

- Using the (weighted sum), we can write

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

Evaluation of error-function derivatives

- Substituting the above two eqs into (chain), we then obtain

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i \qquad \text{(chain*)}$$

- Eq (chain*) tells us that **the required derivative** is obtained simply by multiplying **the value of** δ for the unit at the output end of the weight by **the value of** z for the unit at the input end of the weight (where $z = 1$ in the case of a bias).
- Note that this takes **the same form as** for the simple **linear model** considered at the start of this section. Thus, in order to evaluate the derivatives, we need only to calculate the value of δ_j for each hidden and output unit in the network, and then apply (chain*).

Evaluation of error-function derivatives

- For the output units, we have

$$\delta_k = y_k - t_k \qquad \text{(error)}$$

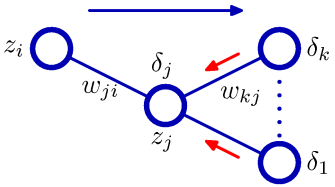
- provided we are using the canonical link as the output-unit activation function.
- To evaluate the δ's for hidden units, we again make use of the chain rule for partial derivatives

$$\delta_j \triangleq \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

  where the sum runs over all units k to which unit j sends connections.

- Units labelled k could include other hidden units and/or output units.

Evaluation of error-function derivatives



$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

- Illustration of the calculation of δ_j for hidden unit j by **backpropagation** of the δ's from those units k to which unit j sends connections.
- The blue arrow denotes the direction of information flow during forward propagation,
- and the red arrows indicate the backward propagation of error information.

Evaluation of error-function derivatives

- If we now substitute the definition of δ into the previous eq., and make use of eqs. (weighted sum) and (chain), we obtain the following **backpropagation** formula

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k \qquad \text{(backpropagation)}$$

- which tells us that the value of δ for a particular hidden unit can be obtained by propagating the δ's backwards from units higher up in the network.
- The summation in (backpropagation) is taken over the **first index** on w_{kj} (corresponding to backward propagation of information through the network),

Evaluation of error-function derivatives

- whereas in the forward propagation eq. (forward) it is taken over the **second index**.
- Because we already know the values of the δ's for the output units, it follows that by recursively applying eq. (backpropagation) we can evaluate the δ's for all of the hidden units in a feed-forward network, regardless of its topology.

Evaluation of error-function derivatives

- The **error backpropagation** procedure can therefore be summarized as follows.

1. Apply an input vector $\mathbf{x}_n$ to the network and forward propagate through the network using (weighted sum) and (activation) to find the activations of all the hidden and output units.
2. Evaluate the δ_k for all the output units using eq. (error).
3. Backpropagate the δ's using (backpropagation) to obtain δ_j for each hidden unit in the network.
4. Use (chain*) to evaluate the required derivatives.

Evaluation of error-function derivatives

- For batch methods, the derivative of the total error E can then be obtained by repeating the above steps for each pattern in the training set and then summing over all patterns:

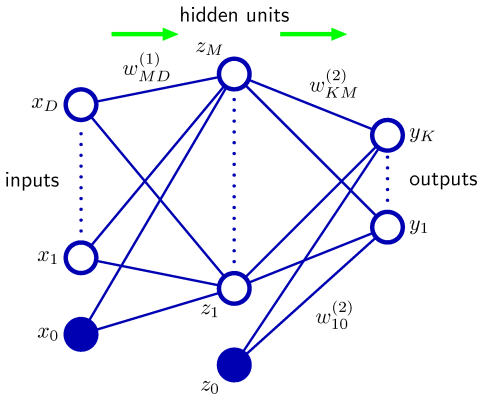
$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}$$

  - In the above derivation we have implicitly assumed that each hidden or output unit in the network has the same activation function $h(\cdot)$.
  - The derivation is easily generalized, however, to allow different units to have individual activation functions, simply by keeping track of which form of $h(\cdot)$ goes with which unit.

Error Backpropagation: An Example

- The above derivation of the backpropagation procedure allowed for general forms for the error function, the activation functions, and the network topology.
- To understand this algorithm well, let us consider a particular example.
- This is chosen both for its simplicity and for its practical importance.

Error Backpropagation: An Example



- A two-layer (three-layer) network of the form illustrated in the figure right.
- Its output units have linear activation functions, $y_k = a_k$
- Its hidden units have hyperbolic tangent activation functions given by

$$h(a) \triangleq \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}$$

- It is with a standard sum-of-squares error function, so that

$$E_n = \frac{1}{2}\sum_{k=1}^{K}(y_k - t_k)^2$$

Error Backpropagation: An Example

- A useful feature of this function is that its derivative can be expressed in the form:

$$h'(a) = 1 - h(a)^2.$$

- For each pattern in the training set in turn, we first perform a forward propagation using

$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i$$

$$z_j = \tanh(a_j)$$

$$y_k = \sum_{j=0}^{M} w_{kj}^{(2)} z_j$$

Error Backpropagation: An Example

- Next we compute the δ's for each output unit using

$$\delta_k = y_k - t_k.$$

- Then we backpropagate these to obtain δ's for the hidden units using

$$\delta_j = (1 - z_j^2) \sum_{k=1}^{K} w_{kj} \delta_k$$

- Finally, the derivatives w.r.t. the first-layer and second-layer weights are given by

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i, \qquad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

Efficiency of backpropagation

- One of the most important aspects of backpropagation is its **computational efficiency**.
- To understand this, let us examine how **the number of computer operations** required to evaluate the derivatives of the error function **scales** with **the total number** W **of weights and biases** in the network.
- A single evaluation of the error function (for a given input pattern) would require $O(W)$ operations, for sufficiently large W.

Efficiency of backpropagation

- Except for a network with very sparse connections, **the number of weights** is typically **much greater** than **the number of units**,
  - so the bulk of the computational effort in forward propagation is concerned with evaluating the sums in eq. (weighted sum),
  - with the evaluation of the activation functions representing a small overhead.
- Each term in the sum in (weighted sum) requires one multiplication and one addition, leading to an overall computational cost that is $O(W)$.

Efficiency of backpropagation

- An alternative approach to backpropagation for computing the derivatives of the error function is to use finite differences.
- This can be done by perturbing each weight in turn with $\varepsilon \ll 1$, and approximating the derivatives by the expression

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \varepsilon) - E_n(w_{ji})}{\varepsilon} + O(\varepsilon) \qquad \text{(Newton's difference quotient)}$$

- In a software simulation, the accuracy of the approximation to the derivatives can be improved by making ε smaller, until numerical roundoff problems arise.

Efficiency of backpropagation

- The accuracy of the finite differences method can be improved significantly by using symmetrical **central differences** of the form

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \varepsilon) - E_n(w_{ji} - \varepsilon)}{2\varepsilon} + O(\varepsilon^2)$$

- In this case, the $O(\varepsilon)$ corrections cancel, as can be verified by Taylor expansion, and so the residual corrections are $O(\varepsilon^2)$.
- The number of computational steps is, however, roughly doubled compared with (Newton's difference quotient).

Efficiency of backpropagation

- The main problem with numerical differentiation is that the highly desirable $O(W)$ scaling has been lost.
  - Each forward propagation requires $O(W)$ steps,
  - and there are W weights in the network each of which must be perturbed individually, so that the overall scaling is $O(W^2)$.

Efficiency of backpropagation

- However, numerical differentiation plays an important role in practice,
- because a comparison of the derivatives calculated by backpropagation with those obtained using central differences provides a powerful check on the correctness of any software implementation of the backpropagation algorithm. (**gradient checking**)

When training networks in practice, derivatives should be evaluated using backpropagation, because this gives the greatest **accuracy** and **numerical efficiency**.

Regularization in NN

Regularization in Neural Networks

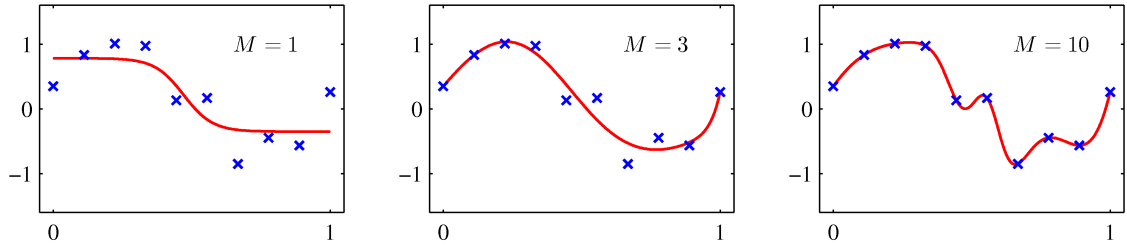
- The number of input and outputs units in a neural network is generally determined by the dimensionality of the data set,
- whereas the number M of hidden units is a free parameter that can be adjusted to give the best predictive performance.
- Note that M controls the number of parameters (weights and biases) in the network,
- and so we might expect that in a maximum likelihood setting there will be **an optimum value** of M that gives **the best generalization performance**,
- i.e., the optimum balance between under-fitting and over-fitting.

Regularization in Neural Networks

- Examples of two-layer networks trained on 10 data points drawn from the sinusoidal data set.
- The graphs show the result of fitting networks having $M = 1, 3$ and 10 hidden units, respectively, by minimizing a sum-of-squares error function using a scaled conjugate-gradient algorithm.

Regularization in Neural Networks

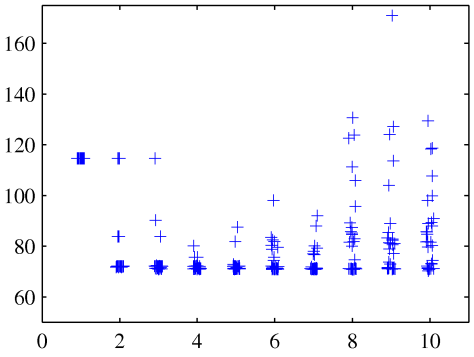
- An example of the effect of different values of M for the sinusoidal regression problem.



Regularization in Neural Networks

- The generalization error, however, is not a simple function of M due to the presence of local minima in the error function.
- Let us see the effect of choosing multiple random initializations for weight vector for a range of values of M, where the overall best validation set performance occurred at $M = 8$.

Regularization in Neural Networks



- Plot of the sum-of-squares test-set error for the polynomial data set versus the number of hidden units in the network, with 30 random starts for each network size, showing the effect of local minima.
- For each new start, the weight vector was initialized by sampling from an isotropic Gaussian distribution having a mean of zero and a variance of 10.

Regularization in Neural Networks

- An alternative is to choose a relatively large value for M and then to control complexity by the addition of **a regularization term** to the error function.
- The simplest regularizer is the quadratic, giving a regularized error

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^{T}\mathbf{w}.$$

- This regularizer is also known as **weight decay**.
- The effective model complexity is then determined by the regularization coefficient λ.
- this regularizer can be interpreted as the negative logarithm of a zero-mean Gaussian prior distribution over the weight vector w.

Consistent Gaussian priors

- One of the limitations of simple weight decay is that is **inconsistent with certain scaling properties** of network mappings.
- Consider a MLP that maps $\{x_i\}$ to $\{y_k\}$.
- The activations of the hidden units in the first hidden layer take the form

$$z_j = h\left(\sum_i w_{ji} x_i + w_{j0}\right)$$

- while the activations of the output units are given by

$$y_k = \sum_j w_{kj} z_j + w_{k0}$$

Consistent Gaussian priors

- Suppose we perform a linear transformation of the input data of the form

$$x_i \longrightarrow \tilde{x}_i = a x_i + b.$$

- Then we can arrange for the mapping performed by the network to be unchanged by making a corresponding linear transformation of the weights and biases from the inputs to the units in the hidden layer of the form

$$w_{ji} \longrightarrow \tilde{w}_{ji} = \frac{1}{a} w_{ji}$$

$$w_{j0} \longrightarrow \tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji}$$

Consistent Gaussian priors

- Similarly, a linear transformation of the output variables of the network of the form

$$y_k \rightarrow \tilde{y}_k = cy_k + d$$

- can be achieved by making a transformation of the second-layer weights and biases using

$$w_{kj} \rightarrow \tilde{w}_{kj} = cw_{kj}$$
$$w_{k0} \rightarrow \tilde{w}_{k0} = cw_{k0} + d$$

Consistent Gaussian priors

- If we train one network using the original data and one network using data for which the input and/or target variables are transformed by one of the above linear transformations,
- then consistency requires that we should obtain equivalent networks that differ only by the linear transformation of the weights as given.
- Any regularizer should be consistent with this property, otherwise it arbitrarily favours one solution over another, equivalent one.
- Clearly, simple weight decay, that treats all weights and biases on an equal footing, does not satisfy this property.

Consistent Gaussian priors

- Look for a regularizer which is invariant under the four linear transformations
- Regularizer should be invariant to re-scaling of the weights and to shifts of the biases.
- Such a regularizer is given by

$$\frac{\lambda_1}{2} \sum_{w \in \mathcal{W}_1} w^2 + \frac{\lambda_2}{2} \sum_{w \in \mathcal{W}_2} w^2 \qquad \text{(improper regularizer)}$$

- where $\mathcal{W}_1$ denotes the set of weights in the first layer, $\mathcal{W}_2$ denotes the set of weights in the second layer, and biases are excluded.
- This regularizer remains unchanged under the weight transformations where parameters are re-scaled using $\lambda_1 \to a^{1/2}\lambda_1$ and $\lambda_2 \to a^{-1/2}\lambda_2$.

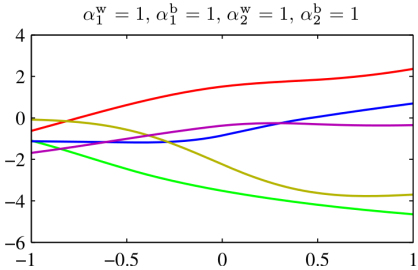
Consistent Gaussian priors

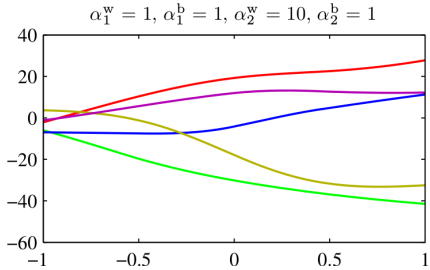
- The (improper regularizer) corresponds to a prior of the form

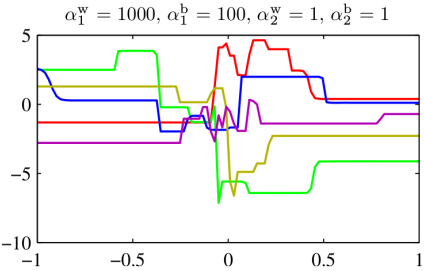
$$p(\mathbf{w}|\alpha_1, \alpha_2) \approx \exp\left(-\frac{\alpha_1}{2}\sum_{w\in\mathcal{W}_1} w^2 - \frac{\alpha_2}{2}\sum_{w\in\mathcal{W}_2} w^2\right)$$

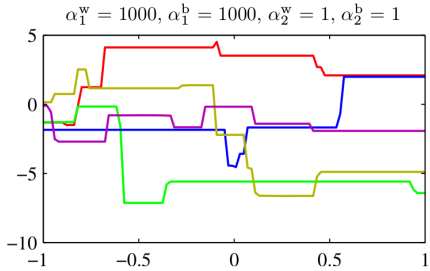
- Priors of this form are improper (they cannot be normalized) because the bias parameters are unconstrained.
- The use of improper priors can lead to difficulties in selecting regularization coefficients and in model comparison within the Bayesian framework, because the corresponding evidence is zero.
- It is therefore common to include separate priors for the biases (which then break shift invariance) having their own hyperparameters.

Consistent Gaussian priors









Consistent Gaussian priors

- We can illustrate the effect of the resulting four hyperparameters by drawing samples from the prior and plotting the corresponding network functions.
- Illustration of the effect of the hyperparameters governing the prior distribution over weights and biases in a two-layer network having a single input, a single linear output, and 12 hidden units having 'tanh' activation functions.
- The priors are governed by four hyperparameters α_1^b, α_1^w, α_2^b, and α_2^w, which represent the precisions of the Gaussian distributions of the first-layer biases, first-layer weights, second-layer biases, and second-layer weights, respectively.

Consistent Gaussian priors

- α_2^w governs the vertical scale of functions (note the different vertical axis ranges on the top two diagrams),
- α_1^w governs the horizontal scale of variations in the function values,
- and α_1^b governs the horizontal range over which variations occur.
- The parameter α_2^b (not illustrated here), governs the range of vertical offsets of the functions.

Consistent Gaussian priors

- More generally, we can consider priors in which the weights are divided into any number of groups $\mathcal{W}_k$ so that

$$p(\mathbf{w}) \approx \exp\left(-\frac{1}{2}\sum_k \alpha_k \|\mathbf{w}\|_k^2\right)$$

- where

$$\|\mathbf{w}\|_k^2 = \sum_{j \in \mathcal{W}_k} w_j^2.$$

- As a special case of this prior, if we choose the groups to correspond to the sets of weights associated with each of the input units, and we optimize the marginal likelihood w.r.t. the corresponding parameters α_k , we obtain **automatic relevance determination**.

Early stopping

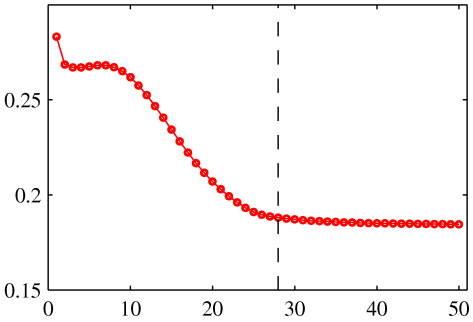
- An alternative to regularization as a way of controlling the effective complexity of a network is the procedure of **early stopping**.
- The training of nonlinear network models corresponds to an iterative reduction of the error function defined w.r.t. a set of training data.
- For many of the optimization algorithms used for network training, such as **conjugate gradients**, the error is a **nonincreasing** function of the iteration index.

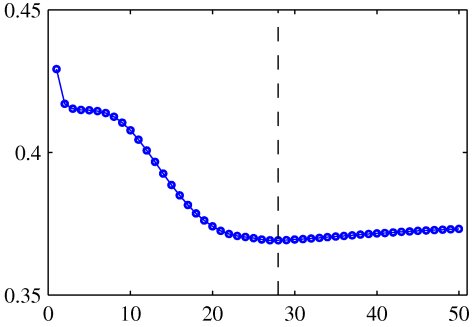
Early stopping

- However, the error measured w.r.t. independent data, generally called a validation set, often shows a **decrease at first**, followed by an **increase as the network starts to over-fit**.
- Training can be stopped at the point of smallest error w.r.t. the validation data set
- in order to obtain a network having good generalization performance.

Early stopping: Training vs Validation error

- The goal of achieving the **best generalization performance** suggests that training should be stopped at the point shown by the vertical dashed lines, corresponding to the **minimum of the validation set error**.

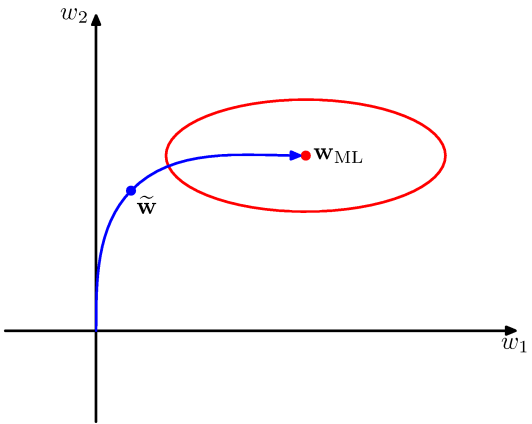




Early stopping

- The behaviour of the network in this case is sometimes explained qualitatively in terms of the effective number of degrees of freedom in the network,
- in which this number starts out small and then to grows during the training process,
- corresponding to a steady increase in the effective complexity of the model.
- Halting training before a minimum of the training error has been reached then represents a way of limiting the effective network complexity.

Early stopping



- In the case of **a quadratic error function**, we can verify this insight, and show that **early stopping** should exhibit similar behaviour to regularization using a simple **weight-decay** term.
- By stopping training early, a weight vector w is found that is qualitatively similar to that obtained with a simple weight-decay regularizer

Invariances

- In many applications of pattern recognition, it is known that predictions should be unchanged, or **invariant**, under one or more **transformations of the input variables**.
- For example, in the classification of objects in two-dimensional images, such as handwritten digits,
- a particular object should be assigned the same classification irrespective of its position within the image (**translation invariance**) or of its size (**scale invariance**).

Invariances

- Such transformations produce significant changes in the raw data, expressed in terms of the intensities at each of the pixels in the image, and yet should give rise to the same output from the classification system.
- Similarly in speech recognition, small levels of nonlinear warping along the time axis, which preserve temporal ordering, should not change the interpretation of the signal.

Invariances

- If **sufficiently large** numbers of training patterns are available, then an adaptive model such as a neural network **can learn the invariance**, at least approximately.
- This involves including within the training set a sufficiently large number of examples of the effects of the **various transformations**.
- Thus, for translation invariance in an image, the training set should include examples of **objects at many different positions**.

Invariances

- This approach may be impractical, however, if the number of training examples is limited, or if there are several invariants
- because the number of combinations of transformations grows exponentially with the number of such transformations.
- We therefore seek alternative approaches for encouraging an adaptive model to exhibit the required invariances.

Invariances

These can broadly be divided into four categories:

1. The training set is **augmented** using replicas of the training patterns, transformed according to the desired invariances. For instance, in our digit recognition example, we could make multiple copies of each example in which the digit is shifted to a different position in each image.

2. A **regularization** term is added to the error function that **penalizes changes in the model output** when the input is transformed. This leads to the technique of **tangent propagation**.

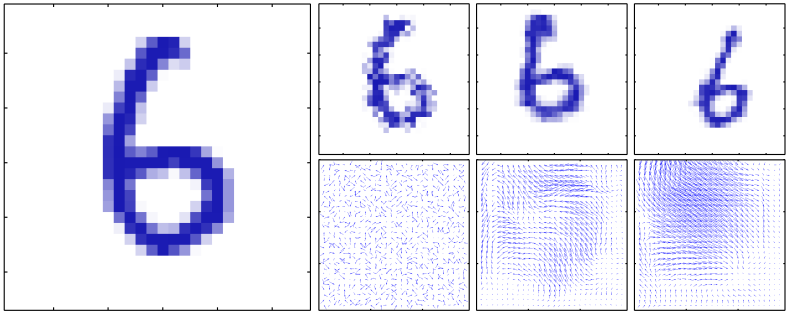
Invariances

3. Invariance is built into the **pre-processing** by extracting features that are invariant under the required transformations. Any subsequent regression or classification system that uses such features as inputs will necessarily also respect these invariances.

4. The final option is to **build the invariance properties into the structure of a neural network** (or into the definition of a kernel function in the case of techniques such as the relevance vector machine). One way to achieve this is through the use of **local receptive fields** and **shared weights**, as discussed in the context of **convolutional neural networks**.

Invariances

- Approach 1 is often relatively easy to implement and can be used to encourage complex invariances.



- Approach 2 leaves the data set unchanged but modifies the error function through the addition of a regularizer.

Invariances

- One advantage of approach 3 is that it can correctly extrapolate well beyond the range of transformations included in the training set.
  - However, it can be difficult to find hand-crafted features with the required invariances that do not also discard information that can be useful for discrimination.
- For **sequential training algorithms**, this can be done by transforming each input pattern before it is presented to the model so that, if the patterns are being recycled, a different transformation (drawn from an appropriate distribution) is added each time.

Invariances

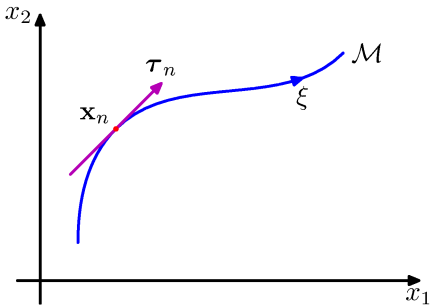
- For **batch methods**, a similar effect can be achieved by replicating each data point a number of times and transforming each copy independently.

The use of such augmented data can lead to significant improvements in generalization (Simard et al., 2003), although it can also be computationally costly.

Tangent propagation

- We can use regularization to encourage models to be invariant to transformations of the input through the technique of **tangent propagation**.
- Consider the effect of a transformation on a particular input vector $\mathbf{x}_n$.
- Provided the transformation is continuous (such as translation or rotation, but not mirror reflection for instance),
- then the transformed pattern will sweep out a manifold $\mathcal{M}$ within the D-dimensional input space.

Tangent propagation



- Illustration of a two-dimensional input space showing the effect of a continuous transformation
- A one-dimensional transformation, parameterized by the continuous variable ξ, applied to $\mathbf{x}_n$ causes it to sweep out a one-dimensional manifold $\mathcal{M}$.
- Locally, the effect of the transformation can be approximated by the tangent vector τ_n.

Tangent propagation

- Consider the case of $D = 2$ for simplicity.
- Suppose the transformation is governed by a single parameter ξ (which might be rotation angle for instance).
- Then the subspace $\mathcal{M}$ swept out by $\mathbf{x}_n$ will be one-dimensional, and will be parameterized by ξ.
- Let the vector that results from acting on $\mathbf{x}_n$ by this transformation be denoted by $\mathbf{s}(\mathbf{x}_n, \xi)$,
- which is defined so that $\mathbf{s}(\mathbf{x}, 0) = \mathbf{x}$.

Tangent propagation

- Then the tangent to the curve $\mathcal{M}$ is given by the directional derivative $\tau = \partial \mathbf{s}/\partial \xi$, and the tangent vector at the point $\mathbf{x}_n$ is given by

$$\boldsymbol{\tau}_n = \left. \frac{\partial \mathbf{s}(\mathbf{x}_n, \xi)}{\partial \xi} \right|_{\xi=0}$$

- Under a transformation of the input vector, the network output vector will, in general, change.

- The derivative of output k w.r.t. ξ is given by

$$\left. \frac{\partial y_k}{\partial \xi} \right|_{\xi=0} = \sum_{i=1}^{D} \frac{\partial y_k}{\partial x_i} \left. \frac{\partial x_i}{\partial \xi} \right|_{\xi=0} = \sum_{i=1}^{D} J_{ki} \tau_i$$

- where J_{ki} is the (k, i) element of the Jacobian matrix $\mathbf{J}$.

Tangent propagation

- The result can be used to modify the standard error function, so as to encourage local invariance in the neighbourhood of the data points, by the addition to the original error function E of a regularization function Ω to give a total error function of the form

$$\tilde{E} = E + \lambda\Omega$$

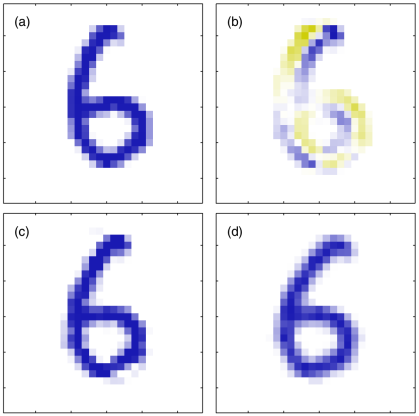
- where λ is a regularization coefficient and

$$\Omega = \frac{1}{2}\sum_{n,k}\left(\left.\frac{\partial y_{nk}}{\partial \xi}\right|_{\xi=0}\right)^2 = \frac{1}{2}\sum_{n,k}\left(\sum_{i=1}^{D} J_{nki}\tau_{ni}\right)^2$$

Tangent propagation

- The regularization function will be zero when the network mapping function is invariant under the transformation in the neighbourhood of each pattern vector,
- and the value of the parameter λ determines the balance between fitting the training data and learning the invariance property.
- The tangent vector $\boldsymbol{\tau}_n$ can be approximated using finite differences,
- by subtracting the original vector $\mathbf{x}_n$ from the corresponding vector after transformation using a small value of ξ, and then dividing by ξ.

Tangent propagation



(a) the original image $\mathbf{x}$ of a handwritten digit,
(b) the tangent vector $\boldsymbol{\tau}$ corresponding to an infinitesimal clockwise rotation,
(c) the result of adding a small contribution from the tangent vector to the original image giving $\mathbf{x} + \varepsilon\boldsymbol{\tau}$ with $\varepsilon = 15$ degrees,
(d) the true image rotated for comparison.

Tangent propagation

- The regularization depends on the network weights through the Jacobian $\mathbf{J}$.
- A backpropagation formalism for computing the derivatives of the regularizer w.r.t. the network weights is easily obtained by extension of the backpropagation.
- If the transformation is governed by L parameters
  - $L = 3$ for translations combined with in-plane rotations in a 2D image,
- then the manifold $\mathcal{M}$ will have dimensionality L, and the corresponding regularizer is given by the sum of terms like Ω, one for each transformation.

Tangent propagation

- If several transformations are considered at the same time, and the network mapping is made invariant to each separately, then it will be (locally) invariant to combinations of the transformations.
- A related technique, called **tangent distance**, can be used to build invariance properties into distance-based methods.

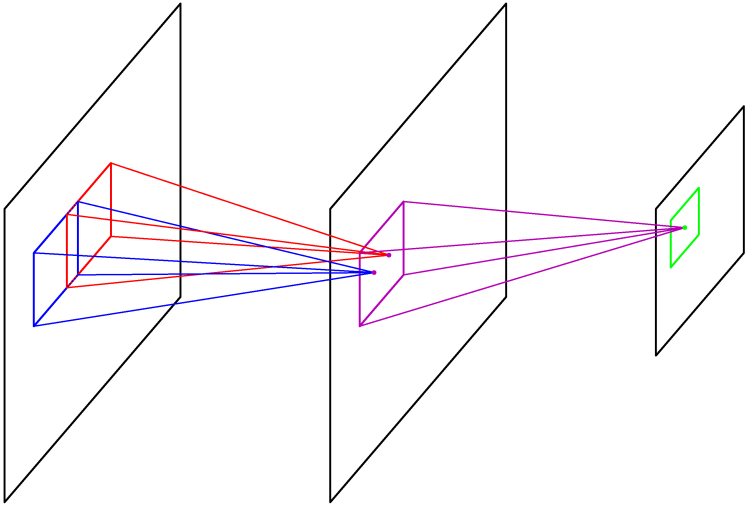
Training with transformed data

- One way to encourage invariance of a model to a set of transformations is to expand the training set using transformed versions of the original input patterns.

Convolutional networks

- Another approach to creating models that are invariant to certain transformation of the inputs is to **build the invariance properties into the structure of a neural network**.
- This is the basis for the **convolutional neural network** (Le Cun et al., 1989; LeCun et al., 1998), which has been widely applied to **image** data.

Convolutional networks



Input image

Convolutional layer

Sub-sampling layer

Convolutional networks

- Diagram illustrating part of a convolutional neural network, showing a layer of convolutional units followed by a layer of subsampling units.
- Several successive pairs of such layers may be used.

Generalization of BP

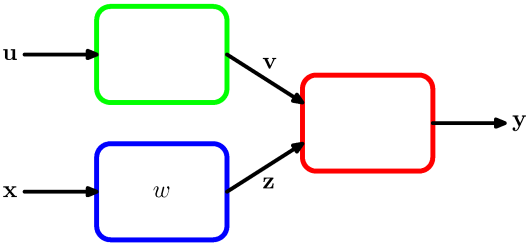
The Jacobian matrix

- The technique of backpropagation can also be applied to the calculation of other derivatives.
- Here we consider the evaluation of the **Jacobian** matrix, whose elements are given by the derivatives of the network outputs w.r.t. the inputs

$$J_{ki} \triangleq \frac{\partial y_k}{\partial x_i}$$

- where each such derivative is evaluated with all other inputs held fixed.

The Jacobian matrix



- Jacobian matrices play a useful role in systems built from a number of distinct modules.
- Each module can comprise a fixed or adaptive function, which can be linear or nonlinear, so long as it is differentiable.

- Suppose we wish to minimize an error function E w.r.t. the parameter w.
- The derivative of the error function is given by

$$\frac{\partial E}{\partial w} = \sum_{k,j} \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial z_j} \frac{\partial z_j}{\partial w}$$

- in which the Jacobian matrix for the **red** module appears in the middle term

The Jacobian matrix

- Because the Jacobian matrix provides a measure of the local sensitivity of the outputs to changes in each of the input variables
- it also allows any known errors Δx_i associated with the inputs to be propagated through the trained network in order to estimate their contribution Δy_k to the errors at the outputs, through the relation

$$\Delta y_k \approx \sum_i \frac{\partial y_k}{\partial x_i} \Delta x_i$$

- which is valid provided the $|\Delta x_i|$ are small.
- In general, the network mapping is nonlinear, and so the elements of the Jacobian matrix will not be constants but will depend on the particular input vector used.

The Jacobian matrix

- The Jacobian matrix can be evaluated using a backpropagation procedure.
- Let us start by writing the element J_{ki} in the form

$$J_{ki} = \frac{\partial y_k}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial a_j} \frac{\partial a_j}{\partial x_i}$$
$$= \sum_j w_{ji} \frac{\partial y_k}{\partial a_j}$$

- The sum runs over all units j to which the input unit i sends connections.
- e.g., over all units in the first hidden layer in the layered topology considered earlier.

The Jacobian matrix

- We now write down a recursive backpropagation formula to determine the derivatives $\frac{\partial y_k}{\partial a_j}$

$$\begin{aligned}\frac{\partial y_k}{\partial a_j} &= \sum_l \frac{\partial y_k}{\partial a_l}\frac{\partial a_l}{\partial a_j} \\ &= h'(a_j)\sum_l w_{lj}\frac{\partial y_k}{\partial a_l}\end{aligned}$$

- where the sum runs over all units l to which unit j sends connections.
- This backpropagation starts at the output units for which the required derivatives can be found directly from the functional form of the output-unit activation function.

The Jacobian matrix

- For instance, if we have individual sigmoidal activation functions at each output unit, then

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj}\sigma'(a_j)$$

- whereas for softmax outputs we have

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj}y_k - y_k y_j$$

The Jacobian matrix

We can summarize the procedure for evaluating the Jacobian matrix as follows.

- Apply the input vector corresponding to the point in input space at which the Jacobian matrix is to be found
- and forward propagate in the usual way to obtain the activations of all of the hidden and output units in the network.
- Next, for each row k of the Jacobian matrix, corresponding to the output unit k, backpropagate using the recursive relation for all of the hidden units in the network.
- Finally, do the backpropagation to the inputs.

The Hessian Matrix

- We have shown how the technique of backpropagation can be used to obtain the first derivatives of an error function w.r.t. the weights in the network.
- Backpropagation can also be used to evaluate the second derivatives of the error, given by

$$\frac{\partial^2 E}{\partial w_{ji} \partial w_{lk}}$$

- It is convenient to consider all of the weight and bias parameters as elements w_i of a single vector, denoted $\mathbf{w}$.
- the second derivatives form the elements H_{ij} of the **Hessian** matrix $\mathbf{H}$, where $i, j \in \{1, \ldots, W\}$ and W is the total number of weights and biases.

The Hessian Matrix

- The Hessian plays an important role in many aspects of neural computing, including the following:

1. Several nonlinear optimization algorithms used for training neural networks are based on considerations of the second-order properties of the error surface, which are controlled by the Hessian matrix
2. The Hessian forms the basis of a fast procedure for re-training a feed-forward network following a small change in the training data
3. The inverse of the Hessian has been used to identify the least significant weights in a network as part of network "pruning" algorithms
4. The Hessian plays a central role in the Laplace approximation for a Bayesian neural network. Its inverse is used to determine the predictive distribution for

The Hessian Matrix: Diagonal approximation

- Some of the applications for the Hessian matrix discussed above require the inverse of the Hessian, rather than the Hessian itself.
- For this reason, there has been some interest in using a diagonal approximation to the Hessian,
- in other words one that simply replaces the off-diagonal elements with zeros, because its inverse is trivial to evaluate.
- We still consider an error function $E = \sum_n E_n$
- The Hessian can then be obtained by considering one pattern at a time, and then summing the results over all patterns.

The Hessian Matrix: Diagonal approximation

- The diagonal elements of the Hessian, for pattern n, can be written

$$\frac{\partial^2 E_n}{\partial w_{ji}^2} = \frac{\partial^2 E_n}{\partial a_j^2} z_i^2.$$

- the 2nd derivatives on the right-hand side of the above equation can be found recursively using the chain rule to give a backpropagation equation of the form

$$\frac{\partial^2 E_n}{\partial a_j^2} = h'(a_j)^2 \sum_{k,k'} w_{kj} w_{k'j} \frac{\partial^2 E_n}{\partial a_k \partial a_{k'}} + h''(a_j) \sum_k w_{kj} \frac{\partial E_n}{\partial a_k}$$

- If we now neglect off-diagonal elements in the second-derivative terms, we obtain

$$\frac{\partial^2 E_n}{\partial a_j^2} = h'(a_j)^2 \sum w_{kj}^2 \frac{\partial^2 E_n}{\partial a_j^2} + h''(a_j) \sum w_{kj} \frac{\partial E_n}{\partial a_j}$$

The Hessian Matrix: Outer product approximation

- When neural networks are applied to regression problems, it is common to use a sum-of-squares error function of the form

$$E = \frac{1}{2}\sum_{n=1}^{N}(y_n - t_n)^2$$

- where we have considered the case of a single output in order to keep the notation simple (the extension to several outputs is straightforward).
- We can then write the Hessian matrix in the form

$$\mathbf{H} = \nabla\nabla E = \sum_{n=1}^{N}\nabla y_n \nabla y_n^{\mathrm{T}} + \sum_{n=1}^{N}(y_n - t_n)\nabla\nabla y_n \qquad \text{(outer product)}$$

- If the network has been trained on the data set, and its outputs y_n happen to be very close to the target values t_n, then the second term will be small

The Hessian Matrix: Outer product approximation

- More generally, however, it may be appropriate to neglect this term by the following argument.
- Recall that the optimal function that minimizes a sum-of-squares loss is the conditional average of the target data.
- The quantity $(y_n - t_n)$ is then a random variable with zero mean.
- If we assume that its value is uncorrelated with the value of the second derivative term on the right-hand side of (outer product),
- then the whole term will average to zero in the summation over n.

The Hessian Matrix: Outer product approximation

- By neglecting the second term in (outer product), we arrive at the Levenberg-Marquardt approximation or outer product approximation, given by

$$\mathbf{H} \approx \sum_{n=1}^{N} \mathbf{b}_n \mathbf{b}_n^{\mathrm{T}}$$

- where $\mathbf{b}_n = \nabla y_n = \nabla a_n$ because the activation function for the output units is simply the identity.
- Evaluation is straightforward as it only involves first derivatives of the error function, which can be evaluated efficiently in $O(W)$ steps using standard backpropagation.
- The elements of the matrix can then be found in $O(W^2)$ steps by simple

The Hessian Matrix: Outer product approximation

- In the case of the **cross-entropy error function** for a network with logistic sigmoid output-unit activation functions,
- the corresponding approximation is given by

$$\mathbf{H} \approx \sum_{n=1}^{N} y_n(1 - y_n)\mathbf{b}_n\mathbf{b}_n^{\mathrm{T}}$$

- An analogous result can be obtained for multiclass networks having softmax output-unit activation functions.

The Hessian Matrix: Inverse Hessian

- We can use the outer-product approximation to develop a computationally efficient procedure for approximating the inverse of the Hessian.
- First we write the outer-product approximation in matrix notation as

$$\mathbf{H}_N \approx \sum_{n=1}^{N} \mathbf{b}_n \mathbf{b}_n^T$$

- where $\mathbf{b}_n = \nabla_{\mathbf{w}} a_n$ is the contribution to the gradient of the output unit activation arising from data point n.

The Hessian Matrix: Inverse Hessian

- We now derive a sequential procedure for building up the Hessian by including data points one at a time.
- Suppose we have already obtained the inverse Hessian using the first L data points.
- By separating off the contribution from data point $L + 1$, we obtain

$$\mathbf{H}_{L+1} = \mathbf{H}_L + \mathbf{b}_{L+1}\mathbf{b}_{L+1}^{\mathrm{T}}.$$

The Hessian Matrix: Inverse Hessian

- In order to evaluate the inverse of the Hessian, we now consider the matrix identity

$$\left(\mathbf{M} + \mathbf{v}\mathbf{v}^{\mathrm{T}}\right)^{-1} = \mathbf{M}^{-1} - \frac{\left(\mathbf{M}^{-1}\mathbf{v}\right)\left(\mathbf{v}^{\mathrm{T}}\mathbf{M}^{-1}\right)}{1 + \mathbf{v}^{\mathrm{T}}\mathbf{M}^{-1}\mathbf{v}}$$

- which is simply a special case of the Woodbury identity.
- If we now identify $\mathbf{H}_{\mathrm{L}}$ with $\mathbf{M}$ and $\mathbf{b}_{\mathrm{L}+1}$ with $\mathbf{v}$, we obtain

$$\mathbf{H}_{\mathrm{L}+1}^{-1} = \mathbf{H}_{\mathrm{L}}^{-1} - \frac{\mathbf{H}_{\mathrm{L}}^{-1}\mathbf{b}_{\mathrm{L}+1}\mathbf{b}_{\mathrm{L}+1}^{\mathrm{T}}\mathbf{H}_{\mathrm{L}}^{-1}}{1 + \mathbf{b}_{\mathrm{L}+1}^{\mathrm{T}}\mathbf{H}_{\mathrm{L}}^{-1}\mathbf{b}_{\mathrm{L}+1}}$$

- In this way, data points are sequentially absorbed until $\mathrm{L} + 1 = \mathrm{N}$ and the whole data set has been processed.

The Hessian Matrix: Inverse Hessian

- This result therefore represents a procedure for evaluating the inverse of the Hessian using a single pass through the data set.
- The initial matrix $\mathbf{H}_0$ is chosen to be $\alpha\mathbf{I}$, where α is a small quantity,
- so that the algorithm actually finds the inverse of $\mathbf{H} + \alpha\mathbf{I}$.
- The results are not particularly sensitive to the precise value of α.
- Extension of this algorithm to networks having more than one output is straightforward.

We note here that the Hessian matrix can sometimes be calculated indirectly as part of the network training algorithm.

In particular, quasi-Newton nonlinear optimization algorithms gradually build up

The Hessian Matrix: Finite differences

- As in the case of the first derivatives of the error function, we can find the second derivatives by using finite differences, with accuracy limited by numerical precision.
- If we perturb each possible pair of weights in turn, we obtain

$$\frac{\partial^2 E}{\partial w_{ji}\partial w_{lk}} = \frac{1}{4\varepsilon^2}\Big\{E(w_{ji}+\varepsilon, w_{lk}+\varepsilon) - E(w_{ji}+\varepsilon, w_{lk}-\varepsilon) - E(w_{ji}-\varepsilon, w_{lk}+\varepsilon) + E(w_{ji}-\varepsilon, w_{lk}-\varepsilon)\Big\} + O(\varepsilon^2).$$

- Again, by using a symmetrical central differences formulation, we ensure that the residual errors are $O(\varepsilon^2)$ rather than $O(\varepsilon)$.

The Hessian Matrix: Finite differences

- Because there are W^2 elements in the Hessian matrix,
- and because the evaluation of each element requires four forward propagations each needing $O(W)$ operations (per pattern),
- we see that this approach will require $O(W^3)$ operations to evaluate the complete Hessian.
- It therefore has poor scaling properties, although in practice it is very useful as a check on the software implementation of backpropagation methods.

The Hessian Matrix: Finite differences

- A more efficient version of numerical differentiation can be found by applying central differences to the first derivatives of the error function,
- which are themselves calculated using backpropagation. This gives

$$\frac{\partial^2 E}{\partial w_{ji} \partial w_{lk}} = \frac{1}{2\varepsilon} \left\{ \frac{\partial E}{\partial w_{ji}}(w_{ji} + \varepsilon) - \frac{\partial E}{\partial w_{ji}}(w_{ji} - \varepsilon) \right\} + O(\varepsilon^2)$$

- Because there are now only W weights to be perturbed,
- and because the gradients can be evaluated in $O(W)$ steps,
- we see that this method gives the Hessian in $O(W^2)$ operations.

The Hessian Matrix: Exact evaluation

- The Hessian can also **be evaluated exactly**, for a network of arbitrary feed-forward topology, using **extension** of the technique of **backpropagation** used to evaluate first derivatives, which shares many of its desirable features including computational efficiency.
- It can be applied to **any differentiable error function** that can be expressed as a function of the network outputs and to networks having arbitrary differentiable activation functions.
- The number of computational steps needed to evaluate the Hessian scales like $O(W^2)$.

The Hessian Matrix: Exact evaluation

- Here we consider the specific case of a network having two layers of weights, for which the required equations are easily derived.
- We shall use indices i and i′ to denote inputs, indices j and j′ to denoted hidden units, and indices k and k′ to denote outputs.
- We first define

$$\delta_k = \frac{\partial E_n}{\partial a_k} \qquad M_{kk'} \triangleq \frac{\partial^2 E_n}{\partial a_k \partial a_{k'}}$$

- where E_n is the contribution to the error from data point n.

The Hessian Matrix: Exact evaluation

- The Hessian matrix for this network can then be considered in three separate blocks:

1. Both weights in the second layer:

$$\frac{\partial^2 E_n}{\partial w_{kj}^{(2)} \partial w_{k'j'}^{(2)}} = z_j z_{j'} M_{kk'}.$$

2. Both weights in the first layer:

$$\frac{\partial^2 E_n}{\partial w_{ji}^{(1)} \partial w_{j'i'}^{(1)}} = x_i x_{i'} h''(a_{j'}) I_{jj'} \sum_k w_{kj'}^{(2)} \delta_k + x_i x_{i'} h'(a_{j'}) h'(a_j) \sum_{k,k'} w_{kj}^{(2)} w_{k'j'}^{(2)} M_{kk'}$$

3. One weight in each layer:

$$\frac{\partial^2 E_n}{\partial w_{ji}^{(1)} \partial w_{k'j'}^{(2)}} = x_i h'(a_{j'}) \left\{ \delta_k h'(a_{j'}) + z_j \sum \partial w_{k'j'}^{(2)} M_{kk'} \right\}.$$

The Hessian Matrix: Fast multiplication

- For many applications of the Hessian, the quantity of interest is not the Hessian matrix $\mathbf{H}$ itself
- but the product of $\mathbf{H}$ with some vector $\mathbf{v}$.
- The evaluation of the Hessian takes $O(W^2)$ operations, and it also requires storage that is $O(W^2)$.
- The vector $\mathbf{v}^{\mathrm{T}}\mathbf{H}$ that we wish to calculate, however, has only W elements,
- so instead of computing the Hessian as an intermediate step, we can instead try to find an efficient approach to evaluating $\mathbf{v}^{\mathrm{T}}\mathbf{H}$ directly in a way that requires only $O(W)$ operations.

The Hessian Matrix: Fast multiplication

- To do this, we first note that

$$\mathbf{v}^{\mathrm{T}}\mathbf{H} = \mathbf{v}^{\mathrm{T}}\nabla(\nabla \mathrm{E})$$

- We can then write down the standard forward-propagation and backpropagation equations for the evaluation of $\nabla \mathrm{E}$
- and apply the above equation to these equations to give a set of forward-propagation and backpropagation equations for the evaluation of $\mathbf{v}^{\mathrm{T}}\mathbf{H}$.
- This corresponds to acting on the original forward-propagation and backpropagation equations with a differential operator $\mathbf{v}^{\mathrm{T}}\nabla$.
- Pearlmutter (1994) used the notation $\mathcal{R}\{\cdot\}$ to denote the operator $\mathbf{v}^{\mathrm{T}}\nabla$.

The Hessian Matrix: Fast multiplication

- The analysis is straightforward and makes use of the usual rules of differential calculus, together with the result

$$\mathcal{R}\mathbf{w} = \mathbf{v}.$$

- The technique is best illustrated with a simple example, and again we choose a two-layer network, with linear output units and a sum-of-squares error function.
- we consider the contribution to the error function from one pattern in the data set.
- The required vector is then obtained as usual by summing over the contributions from each of the patterns separately.

The Hessian Matrix: Fast multiplication

- For the two-layer network, the forward-propagation equations are given by

$$a_j = \sum_i w_{ji} x_i$$
$$z_j = h(a_j)$$
$$y_k = \sum_j w_{kj} z_j$$

- Apply the $\mathcal{R}\{\cdot\}$ operator on these equations to obtain

$$\mathcal{R}\{a_j\} = \sum_i v_{ji} x_i$$
$$\mathcal{R}\{z_j\} = h'(a_j)\mathcal{R}\{a_j\}$$
$$\mathcal{R}\{y_k\} = \sum_j w_{kj}\mathcal{R}\{z_j\} + \sum_j v_{kj} z_j$$

- where v_{ji} is the element of the vector $\mathbf{v}$ that corresponds to the weight w_{ji}.

The Hessian Matrix: Fast multiplication

- Quantities of the form $\mathcal{R}\{z_j\}$, $\mathcal{R}\{a_j\}$ and $\mathcal{R}\{y_k\}$ are to be regarded as new variables whose values are found using the above equations.
- Because we are considering a sum-of-squares error function, we have the following standard backpropagation expressions:

$$\delta_k = y_k - t_k$$
$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

- Again, we act on these equations with the $\mathcal{R}\{\cdot\}$ operator to obtain a set of backpropagation equations in the form

$$\mathcal{R}\{\delta_k\} = \mathcal{R}\{y_k\}$$
$$\mathcal{R}\{\delta_j\} = h''(a_j)\mathcal{R}\{a_j\} \sum_i w_{kj}\delta_k + h'(a_j) \sum_i v_{kj}\delta_k + h'(a_j) \sum_i w_{kj}\mathcal{R}\{\delta_k\}$$

The Hessian Matrix: Fast multiplication

- Finally, we have the usual equations for the first derivatives of the error

$$\frac{\partial E}{\partial w_{kj}} = \delta_k z_j$$
$$\frac{\partial E}{\partial w_{ji}} = \delta_j x_i$$

- and acting on these with the $\mathcal{R}\{\cdot\}$ operator, we obtain expressions for the elements of the vector $\mathbf{v}^{\mathrm{T}}\mathbf{H}$

$$\mathcal{R}\left\{\frac{\partial E}{\partial w_{kj}}\right\} = \mathcal{R}\{\delta_k\} z_j + \delta_k \mathcal{R}\{z_j\}$$
$$\mathcal{R}\left\{\frac{\partial E}{\partial w_{ji}}\right\} = x_i \mathcal{R}\{\delta_j\}$$

(multiplication)

The Hessian Matrix: Fast multiplication

- The implementation of this algorithm involves the introduction of additional variables
  - $\mathcal{R}\{a_j\}$, $\mathcal{R}\{z_j\}$ and $\mathcal{R}\{\delta_j\}$ for the hidden units
  - and $\mathcal{R}\{\delta_k\}$ and $\mathcal{R}\{y_k\}$ for the output units.
- For each input pattern, the values of these quantities can be found using the above results
- The elements of $\mathbf{v}^{\mathrm{T}}\mathbf{H}$ are then given by (multiplication).
- An elegant aspect of this technique is that the equations for evaluating $\mathbf{v}^{\mathrm{T}}\mathbf{H}$ mirror closely those for standard forward and backward propagation,
- and so the extension of existing software to compute this product is typically

The Hessian Matrix: Fast multiplication

If desired, the technique can be used to evaluate the full Hessian matrix by choosing the vector $\mathbf{v}$ to be given successively by a series of unit vectors of the form

$$(0, 0, ..., 1, ..., 0)$$

each of which picks out one column of the Hessian.